

```
In [ ]: import marimo as mo
```

Sudoku Genérico como CSP

Unidade curricular: Lógica Computacional **Autores:** Miguel do Paço Cruz (a108574) e Pedro Rodrigues Campos (a108482)

1. Introdução

Neste trabalho resolvemos o Sudoku $n^2 \times n^2$ como um problema de satisfação de restrições (CSP). As regras do jogo são todas a mesma restrição: um conjunto de células tem de ter valores diferentes. O que muda entre uma linha, uma coluna e um bloco é só quais as células do conjunto.

Por isso há uma classe única para um grupo de células (`Box`) e o resto é construído a partir dela: linhas e colunas (`Path`), blocos (`Cube`) e pistas aleatórias (outro `Box`). O modelo CSP trata todos os grupos da mesma maneira.

2. Objetivos

- Representar qualquer grupo de células através de uma classe genérica, com algumas células possivelmente fixas.
- Obter linhas, colunas e blocos como especializações dessa classe.
- Gerar aleatoriamente pistas iniciais reutilizando a mesma abstração.
- Montar o modelo completo e resolvê-lo, distinguindo claramente o caso em que não existe solução.
- Validar automaticamente as soluções e garantir que o código funciona para qualquer n , e não apenas para $n = 3$.

3. Correspondência com o enunciado

Enunciado	Neste notebook
<code>box</code> (R1)	<code>Box</code>
<code>cube</code> (R2)	<code>Cube</code>
<code>path</code> (R3)	<code>Path</code>
geração de pistas (R4)	<code>random_clues</code>
modelo e resolução (R5)	<code>SudokuSolver</code> (<code>add_groups</code> , <code>solve</code>)

4. Decisões de modelação

Solver. Usámos o CP-SAT do OR-Tools. O Sudoku é um problema de satisfação com domínios finitos e restrições `AllDifferent`, que o CP-SAT suporta diretamente, por isso o modelo fica igual ao enunciado: uma variável em $[1, n^2]$ por célula e uma restrição por grupo. Não há nada para otimizar.

Pistas. O enunciado manda impor "todos diferentes" a cada grupo que o modelo recebe. Isso está certo para linhas, colunas e blocos, mas não para as pistas: são células soltas pela grelha e duas podem ter o mesmo valor sem problema nenhum se não partilharem linha, coluna ou bloco. Com `AllDifferent` nas pistas, puzzles solúveis davam "sem solução", e com $k > n^2$ davam sempre. Por isso `add_groups` tem o parâmetro `all_different` (por omissão `True`) e as pistas entram com `False`, só a fixar valores.

Puzzles sem solução. Como as pistas são independentes, podem entrar em conflito (o mesmo valor na mesma linha, por exemplo). Quando isso acontece, `solve_full_sudoku` gera novas pistas, até um máximo de `max_tentativas`; se esgotar, devolve `SEM_SOLUCAO`. Já `resolver`, que resolve umas pistas dadas, limita-se a indicar que não há solução.

Estados. `solve` devolve `SOLUCAO`, `SEM_SOLUCAO` (o solver provou que é impossível) ou `DESCONHECIDO` (por exemplo, acabou o tempo), para não confundir "não existe" com "não se sabe".

5. Grupos de células

5.1 Box : grupo genérico (R1)

Guarda `(linha, coluna) → valor` ou `None`. Não tem nada específico de linhas, colunas ou blocos. O `add` rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

```
In [ ]: class Box:

    def __init__(self, n: int, initial_cells: dict | None = None):
        if isinstance(n, bool) or not isinstance(n, int) or n < 1:
            raise ValueError(f"n tem de ser um inteiro positivo, recebido {n!r}.")
        self.n = n
        self.max_val = n * n
        self.cells: dict[tuple[int, int], int | None] = {}

        if initial_cells:
            for (i, j), val in initial_cells.items():
```

```

        self.add(i, j, val)

    def add(self, i: int, j: int, val: int | None = None):
        for x in (i, j):
            if isinstance(x, bool) or not isinstance(x, int):
                raise ValueError(f"Coordenada {x!r} não é um inteiro.")
            if not (0 <= i < self.max_val and 0 <= j < self.max_val):
                raise ValueError(
                    f"Coordenadas ({i}, {j}) fora da grelha {self.max_val}x{self.max_val}."
                )

            if val is not None:
                if isinstance(val, bool) or not isinstance(val, int) or not (1 <= val <= self.max_val):
                    raise ValueError(f"Valor {val!r} fora do intervalo [1, {self.max_val}].")

        self.cells[(i, j)] = val

    def to_matrix(self) -> list[list[int]]:
        matrix = [[0] * self.max_val for _ in range(self.max_val)]
        for (i, j), val in self.cells.items():
            if val is not None:
                matrix[i][j] = val
        return matrix

    def __repr__(self):
        return f"{type(self).__name__}(n={self.n}, total_celulas={len(self.cells)})"

```

5.2 Cube e Path: blocos, linhas e colunas (R2, R3)

Cube é o bloco $n \times n$ de índices (b_i, b_j) . **Path** é um troço reto entre **inicio** e **fim**, em qualquer sentido. Uma linha ou coluna inteira é um **Path** de uma ponta à outra da grelha.

```

In [ ]: class Cube(Box):
        def __init__(self, n: int, bi: int, bj: int, initial_values: dict | None = None):
            super().__init__(n)
            if not (0 <= bi < n and 0 <= bj < n):
                raise ValueError(f"Índices de bloco ({bi}, {bj}) fora do intervalo [0, {n - 1}].")

            initial_values = initial_values or {}
            for di in range(n):
                for dj in range(n):
                    r, c = bi * n + di, bj * n + dj
                    self.add(r, c, initial_values.get((r, c)))

```

```

In [ ]: class Path(Box):
        def __init__(self, n: int, inicio: tuple[int, int], fim: tuple[int, int],

```

```

        initial_values: dict | None = None):
    super().__init__(n)
    r1, c1 = inicio
    r2, c2 = fim

    if r1 != r2 and c1 != c2:
        raise ValueError("Path tem de ser um troço estritamente horizontal ou
vertical.")

    dr = (r2 > r1) - (r2 < r1)
    dc = (c2 > c1) - (c2 < c1)
    comprimento = max(abs(r2 - r1), abs(c2 - c1)) + 1

    initial_values = initial_values or {}
    for k in range(comprimento):
        r, c = r1 + k * dr, c1 + k * dc
        self.add(r, c, initial_values.get((r, c)))

```

6. Geração aleatória de pistas (R4)

`random_clues` devolve um `Box` com k células distintas, cada uma fixa a um valor aleatório em $[1, n^2]$.

```
In [ ]: import random
```

```
In [ ]: def random_clues(n: int, k: int | None = None, rng: random.Random | None = None) -
> Box:
    rng = rng or random
    if k is None:
        k = n

    max_dim = n * n
    total = max_dim * max_dim
    if not (0 <= k <= total):
        raise ValueError(f"k ({k}) tem de estar entre 0 e o número de células
({total}).")

    box = Box(n)
    for idx in rng.sample(range(total), k):
        box.add(idx // max_dim, idx % max_dim, rng.randint(1, max_dim))
    return box

```

7. Modelo CSP e resolução (R5, R6)

`SudokuSolver` tem uma variável por célula e aplica os grupos que recebe. `resolver` e `solve_full_sudoku` juntam linhas, colunas, blocos e pistas e resolvem.

```
In [ ]: from ortools.sat.python import cp_model
```

```

class SudokuSolver:
    SOLUCAO = "SOLUCAO"

```

```

SEM_SOLUCAO = "SEM_SOLUCAO"
DESCONHECIDO = "DESCONHECIDO"

def __init__(self, n: int):
    self.n = n
    self.max_dim = n * n
    self.model = cp_model.CpModel()
    self.grid = [
        [self.model.NewIntVar(1, self.max_dim, f"cell_{r}_{c}") for c in
range(self.max_dim)]
        for r in range(self.max_dim)
    ]

    def add_groups(self, groups: list[Box], all_different: bool = True):
        for box in groups:
            if box.n != self.n:
                raise ValueError(f"Grupo com n={box.n} incompatível com o modelo
(n={self.n}).")
            cell_vars = []
            for (r, c), val in box.cells.items():
                var = self.grid[r][c]
                cell_vars.append(var)
                if val is not None:
                    self.model.Add(var == val)
            if all_different and len(cell_vars) > 1:
                self.model.AddAllDifferent(cell_vars)

    def solve(self, max_time: float = 10.0):
        solver = cp_model.CpSolver()
        solver.parameters.max_time_in_seconds = max_time
        status = solver.Solve(self.model)

        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            solution = [
                [solver.Value(self.grid[r][c]) for c in range(self.max_dim)]
                for r in range(self.max_dim)
            ]
            return self.SOLUCAO, solution
        if status == cp_model.INFEASIBLE:
            return self.SEM_SOLUCAO, None
        return self.DESCONHECIDO, None

```

```

In [ ]: def resolver(n: int, pistas: Box, max_time: float = 10.0):
    max_dim = n * n
    estrutura = []
    for r in range(max_dim):
        estrutura.append(Path(n, (r, 0), (r, max_dim - 1)))
    for c in range(max_dim):
        estrutura.append(Path(n, (0, c), (max_dim - 1, c)))
    for bi in range(n):
        for bj in range(n):
            estrutura.append(Cube(n, bi, bj))

    solver = SudokuSolver(n)
    solver.add_groups(estrutura)
    solver.add_groups([pistas], all_different=False)

```

```

    return solver.solve(max_time)

def solve_full_sudoku(n: int, k_pistas: int | None = None,
                     max_tentativas: int = 50,
                     rng: random.Random | None = None,
                     max_time: float = 10.0):
    pistas = None
    for tentativa in range(1, max_tentativas + 1):
        pistas = random_clues(n, k_pistas, rng)
        estado, solucao = resolver(n, pistas, max_time)
        if estado != SudokuSolver.SEM_SOLUCAO:
            return estado, solucao, pistas, tentativa
    return SudokuSolver.SEM_SOLUCAO, None, pistas, max_tentativas

```

8. Validação e testes

8.1 Validador

`validate_solution` confirma que linhas, colunas e blocos têm os valores $1 \dots n^2$ e que as pistas se mantêm.

```

In [ ]: def validate_solution(n: int, solucao: list[list[int]], pistas: Box) -> bool:
    max_dim = n * n
    esperado = set(range(1, max_dim + 1))

    if len(solucao) != max_dim or any(len(linha) != max_dim for linha in solucao):
        return False

    for r in range(max_dim):
        if set(solucao[r]) != esperado:
            return False

    for c in range(max_dim):
        if {solucao[r][c] for r in range(max_dim)} != esperado:
            return False

    for bi in range(n):
        for bj in range(n):
            bloco = {
                solucao[bi * n + di][bj * n + dj]
                for di in range(n) for dj in range(n)
            }
            if bloco != esperado:
                return False

    for (r, c), val in pistas.cells.items():
        if val is not None and solucao[r][c] != val:
            return False

    return True

```

```

In [ ]: def run_tests():
    def rejeita(f, *args, **kwargs):

```

```

    try:
        f(*args, **kwargs)
    except ValueError:
        return
    raise AssertionError(f"Deveria ter levantado ValueError: {args} {kwargs}")

# Box
b = Box(3)
for i, j in [(9, 0), (0, 9), (-1, 0), (0, -1)]:
    rejeita(b.add, i, j)
for v in [0, 10, -3, True, 2.5]:
    rejeita(b.add, 0, 0, v)
b.add(0, 0, 5)
b.add(1, 1)
m = b.to_matrix()
assert m[0][0] == 5 and m[1][1] == 0 and sum(map(sum, m)) == 5

# Cube
cubo = Cube(3, 1, 2)
assert set(cubo.cells) == {(r, c) for r in range(3, 6) for c in range(6, 9)}
rejeita(Cube, 3, 3, 0)
rejeita(Cube, 3, 0, 3)

# Path
assert set(Path(3, (2, 1), (2, 4)).cells) == {(2, c) for c in range(1, 5)}
assert set(Path(3, (2, 4), (2, 1)).cells) == {(2, c) for c in range(1, 5)}
assert set(Path(3, (0, 3), (8, 3)).cells) == {(r, 3) for r in range(9)}
assert set(Path(3, (8, 3), (0, 3)).cells) == {(r, 3) for r in range(9)}
assert set(Path(3, (4, 4), (4, 4)).cells) == {(4, 4)}
rejeita(Path, 3, (0, 0), (1, 1))
rejeita(Path, 3, (0, 0), (0, 9))

# random_clues
p = random_clues(3, rng=random.Random(0))
assert len(p.cells) == 3
assert all(1 <= v <= 9 for v in p.cells.values())
assert len(random_clues(3, k=20, rng=random.Random(0)).cells) == 20
rejeita(random_clues, 3, 82)

# insolúvel vs. pistas iguais sem relação
conflito = Box(3, {(0, 0): 5, (0, 1): 5})
estado, sol = resolver(3, conflito)
assert estado == SudokuSolver.SEM_SOLUCAO and sol is None

independentes = Box(3, {(0, 0): 5, (4, 4): 5})
estado, sol = resolver(3, independentes)
assert estado == SudokuSolver.SOLUCAO and validate_solution(3, sol,
independentes)

# validador
vazio = Box(3)
_, boa = resolver(3, vazio)
assert validate_solution(3, boa, vazio)
ma = [linha[:] for linha in boa]
ma[0][0], ma[0][1] = ma[0][1], ma[0][0]
assert not validate_solution(3, ma, vazio)

```

```

# fluxo completo
validadas = {}
tentativas_total = {}
for n in (2, 3):
    validadas[n] = 0
    tentativas_total[n] = 0
    for semente in range(20):
        estado, sol, pistas, t = solve_full_sudoku(n,
rng=random.Random(semente))
        assert estado == SudokuSolver.SOLUCAO, f"n={n}, semente={semente}"
        assert validate_solution(n, sol, pistas)
        validadas[n] += 1
        tentativas_total[n] += t

    return validadas, tentativas_total

resultados_testes = run_tests()

```

9. Resolução de um exemplo

Gera-se um puzzle com pistas aleatórias (semente fixa, para ser reproduzível) e resolve-se. Esta secção só calcula; a apresentação fica na secção seguinte.

```

In [ ]: n = 3
k_pistas = 5
semente = 2026

estado, solucao, pistas, tentativas = solve_full_sudoku(
    n, k_pistas, rng=random.Random(semente)
)

_lista = ", ".join(
    f"({r}, {c}) = {v}" for (r, c), v in sorted(pistas.cells.items())
)
mo.md(f"""
- **n:** {n} (grelha {n * n}x{n * n}), **pistas:** {k_pistas}, **semente:**
{semente}
- **Estado:** {estado} ao fim de {tentativas} tentativa(s)
- **Pistas (linha, coluna) = valor**, com índices a partir de 0: {_lista}
""")

```

10. Visualização

A grelha é desenhada como tabela markdown. Os valores a **negrito** são as pistas, **.** é uma célula vazia e **|** e **-** separam os blocos $n \times n$.

```

In [ ]: def render_sudoku_grid(n: int, grid: list[list[int]], pistas: Box | None = None):
    max_dim = n * n
    pistas_dict = pistas.cells if pistas else {}

```

```

def fmt(r, c):
    v = grid[r][c]
    if not v:
        return "."
    return f"***{v}***" if pistas_dict.get((r, c)) is not None else str(v)

def linha(celulas):
    return "|" + " | ".join(celulas) + " |"

colunas = []
for c in range(max_dim):
    if c > 0 and c % n == 0:
        colunas.append(None)
    colunas.append(c)

linhas = [linha(["&nbsp;"] * len(colunas)), linha([":-"] * len(colunas))]
for r in range(max_dim):
    if r > 0 and r % n == 0:
        linhas.append(linha(["†" if c is None else "-" for c in colunas]))
    linhas.append(linha(["| " if c is None else fmt(r, c) for c in colunas]))

return mo.md("\n".join(linhas))

```

```

In [ ]: if solucao is not None:
        saida = mo.vstack([
            mo.md("### Pistas iniciais"),
            render_sudoku_grid(n, pistas.to_matrix(), pistas),
            mo.md("### Solução encontrada pelo CSP"),
            render_sudoku_grid(n, solucao, pistas),
        ])
    else:
        saida = mo.md(f"Sem solução (`{estado}`).")

saida

```

Utilização de IA. Para este trabalho recorreremos ao ChatGPT para obter sugestões e apoio na implementação e revisão do código, bem como para completar e melhorar os comentários e a documentação presentes no notebook. A conversa está disponível [aqui](#). As sugestões obtidas foram analisadas e adaptadas por nós de acordo com os requisitos e decisões tomadas para este trabalho.

Para este trabalho recorreremos ao Gemini para obter sugestões e apoio na implementação e revisão do código, nomeadamente na verificação automática da solução. A conversa está disponível [aqui](#). As sugestões obtidas foram analisadas e adaptadas por nós de acordo com os requisitos e decisões tomadas para este trabalho.